Exploring AI-Assisted Test-Driven Development in Introductory Programming Education

Igor Roberto Michalski de Souza
igorroberto2312@gmail.com
Department of Computer Science, State University of Mato Grosso do Sul (UEMS)
Dourados, Brazil

Felipe Restrepo Calle
ferestrepoca@unal.edu.co
Department of Systems and Industrial Engineering, Universidad Nacional de Colombia (UNAL)
Bogotá, Colombia

Kauan Henrick Teixeira da Silva
kauandugi14@gmail.com
Department of Computer Science, State University of Mato Grosso do Sul (UEMS)
Dourados, Brazil

Victor Rech Vendruscolo
rechvvictor@gmail.com
Department of Computer Science, State University of Mato Grosso do Sul (UEMS)
Dourados, Brazil

Silvana Morita Melo
silvanamelo@ufgd.edu.br
Department of Computer Engineering, Federal University of Grande Dourados (UFGD)
Dourados, Brazil

Jorge Marques Prates
jorgemprates@gmail.com
Department of Computer Science, São Paulo State University (UNESP)
Presidente Prudente, Brazil

ABSTRACT

GenAI tools are rapidly entering introductory programming, yet their role in shaping the test-first discipline of TDD remains underexplored for novices. This exploratory quasi-experiment examines how beginners apply TDD when supported by GenAI, focusing on artefact quality and learner experience. An 8-hour supervised quasi-experiment, with a complementary qualitative component, spanned two short courses (C with assert tests; JavaScript with Jest). We scored code artefacts with a rubric, analysed interaction logs, and collected post-activity surveys, integrating the quantitative and qualitative evidence. Performance was moderate-to-high but variable: correctness was typically intermediate, design/structure higher, and style/readability less consistent. Logs characterise GenAI as a procedural and conceptual scaffold, alongside iterative delegation. GenAI can support early TDD, but requires structured guidance to preserve deliberate test design.

KEYWORDS

Generative Artificial Intelligence, Test-Driven Development, Programming Education, Large Language Models

1 Introduction

Generative Artificial Intelligence (GenAI) tools such as ChatGPT, GitHub Copilot, and Gemini have rapidly become part of the learning ecosystem for novice programmers. These tools provide natural-language explanations, code suggestions, debugging support, and automated test generation, lowering technical barriers that often challenge beginners. As a result, students are increasingly integrating GenAI into their early programming activities, using it both to accelerate implementation tasks and to clarify fundamental concepts [19].

In the context of software engineering education, these concerns become even more relevant when students are exposed to structured development practices such as Test-Driven Development (TDD) [3]. TDD requires disciplined thinking: designing tests before implementation, reasoning about program behavior, and iteratively refining solutions based on failing tests. When GenAI tools are introduced into this process, they may reshape how students approach test design and code quality assurance [2].

Recent studies have explored how students use GenAI tools during programming assignments, reporting benefits such as improved debugging efficiency, reduced syntactic errors, and increased confidence. Reviews such as Prather et al. [19] have shown that GenAI can serve as an explanatory tutor, a provider of examples, and a source of alternative solutions. Other works, however, highlight risks including hallucinations, incomplete reasoning, reduced metacognitive effort, and the temptation for students to adopt AI-generated solutions without critical evaluation [19]. Research on the use of GenAI for test generation also shows mixed results: while LLMs can assist in producing test cases, they often generate incomplete or logically inconsistent tests unless guided by human validation [24].

Empirical studies in computing education have begun to document how students engage with GenAI during learning activities, but evidence remains fragmented. Some case studies show that AI can scaffold novices through conceptual difficulties, while others show contradictory effects, such as decreased problem-solving persistence or over-dependence on model suggestions [7, 8].

Although interest in GenAI-assisted learning is rapidly growing, the role of these tools in supporting foundational programming practices remains an open and underexplored research area. In particular, few empirical studies have characterised how GenAI shapes the process and artefact outcomes of TDD performed by novice programmers, both in terms of code quality and learning experience. Prior studies typically examine isolated aspects, rather than the structured test-first workflow that TDD demands. Understanding these patterns is essential to informing curriculum design, the development of learning scaffolds, and the responsible integration of GenAI into early programming education.

This exploratory quasi-experiment, complemented by a qualitative component, investigates how novice programmers apply the TDD methodology when assisted by a generative AI tool. The objective is to characterise both the technical outcomes and the subjective experience associated with AI-assisted TDD. By combining artefact-based evidence, behaviour traces, and perception data, the study

provides an integrated perspective on how GenAI characterises the development process during introductory programming tasks.

To guide this investigation, we address the following research questions: **RQ1:** What are the technical characteristics of the code produced by novice programmers when applying TDD with generative AI support? **RQ2:** How do novice programmers perceive the utility and challenges of Test-Driven Development when assisted by generative AI?

This study is situated within a broader educational initiative at the State University of Mato Grosso do Sul (UEMS), where structured activities have been conducted to explore the pedagogical role of GenAI in early-stage programming courses. As part of this initiative, an 8 hour short course on Software Testing and TDD was offered to first-year students, combining technical instruction with responsible use of AI tools. The quasi-experiment presented in this paper builds upon this educational experience and investigates how students engage with AI within a controlled, test-first development scenario.

The contributions of this paper are threefold. First, it provides empirical evidence on how novice programmers use GenAI during TDD, integrating behavioural, artefact-based, and perception data. Second, it describes observed patterns in how AI assistance relates to code quality and the cognitive experience of performing TDD. Third, it contributes a structured research protocol and an experimental artefact package that can be reused or adapted by future studies in SQA education.

2 Related Works

The integration of Large Language Models (LLMs) into computing education has prompted a growing body of research examining their pedagogical and technical implications. Within this broader context, particular attention has been given to introductory programming education, where LLMs are increasingly used to support novice learners in tasks such as code generation, explanation, debugging, and testing [9, 20].

Empirical evidence suggests that AI-based support can enhance early programming learning, particularly when it is designed to promote learner autonomy rather than replace students' problem solving. Kim et al. [10] introduced a conversational environment that encouraged students to express and validate ideas through proactive unit tests, effectively embedding Test Driven Development principles into physical computing tasks. Similarly, the DBox system demonstrated that co-decomposition between learner and LLM improves conceptual understanding, emphasizing the value of structured collaboration. In classroom settings, Lyu et al. [12] found that human–AI pairs outperform solo learners with AI, highlighting that LLMs function best as cognitive scaffolds [13].

At the same time, frameworks such as Cognitive-Driven Development (CDD) propose explicit monitoring of intrinsic cognitive complexity (ICPs) to manage mental load during development. This approach parallels the incremental nature of TDD, illustrating that structured reflection whether cognitive or test driven enhances understanding and long term retention. Together, these works indicate that the benefits of AI-assisted learning arise from guided interaction, reflection, and control over automation [6].

Recent large-scale reviews have reinforced these findings. Prather et al. [19] highlight that, despite the rapid adoption of generative AI tools across educational settings, their effectiveness depends heavily on the presence of well-structured guidance, opportunities for self-reflection, and mechanisms that prevent over-reliance on the model. Their review shows that LLMs can accelerate learning and reduce syntactic barriers, but they also introduce risks such as superficial problem-solving, decreased metacognitive effort, and the propagation of subtle errors particularly when students use AI as a substitute rather than a scaffold. These observations align with prior work emphasizing the importance of human-AI collaboration, controlled integration, and pedagogical structure when employing LLMs for programming education.

Test-Driven Development (TDD) is a software development practice in which programmers write tests before implementing the corresponding production code. The process follows the well-known red–green–refactor cycle: first, the developer writes a failing test (red), then produces the simplest code that makes the test pass (green), and finally improves the internal structure of the solution without changing its behaviour (refactor) [3]. TDD promotes incremental design, clearer requirements, and higher software reliability by ensuring that every feature is defined through executable tests from the outset.

In recent years, studies have begun to explicitly connect LLMs to the TDD cycle. Mock et al. [16] showed that generative AI can automate parts of the red-green-refactor loop, such as generating test skeletons and proposing refactorings, though human validation remains essential to prevent false positives. Industrial experiments, like the presented by Ferreira et al. [5], confirm high practical usefulness of LLM generated acceptance tests (92–95% accuracy). Furthermore, Olmez and Gehringer [18] observed significant increases in assertion density among students using AI-generated unit tests. In a TDD-based practice environment for Python beginners, Son and Song [21] compared researcher-designed, GPT-generated, strategy-guided GPT, and student-written test cases, reporting that strategy-guided GPT cases achieved higher fault detection and efficiency than even the researcher-designed ones, and that TDD practice strengthened learners' logical thinking and error recognition.

Comparative analyses across AI tools reveal performance diversity. Muñoz et al. [17] and Manik [14] found that no single model consistently dominates, with ChatGPT and DeepSeek showing trade-offs between accuracy and efficiency. Kumar et al. [11] further demonstrated that while LLMs provide quick answers, students relying solely on them may perform worse in conceptual reasoning tasks compared to traditional web search users. Observing 12 undergraduates writing unit tests with ChatGPT, Ardiç et al. [1] identified strategies ranging from full delegation of test ideas and implementation to mostly manual work; although students valued time savings and reduced cognitive load, they also reported diminished trust and a lack of ownership over generated tests. These findings reinforce the importance of coupling AI with structured testing methodologies like TDD to sustain active reasoning rather than passive completion.

Overall, the literature demonstrates that LLMs can support learning in introductory programming education through scaffolding, testing support, and guided reflection. However, empirical comparisons between traditional TDD and AI-assisted

TDD remain scarce, particularly in introductory programming contexts. This study contributes by descriptively characterising how generative AI shapes testing behaviour, code quality, and students' perceptions of utility and confidence within introductory TDD-based programming activities.

3 Methodology

This study adopts a quasi-experimental design with a complementary qualitative component, grounded in the principles of Empirical Software Engineering described by Wohlin et al. [23]. The methodology combines quantitative evaluation of programming artefacts with qualitative analyses of interaction logs and perception data to characterize how novice programmers apply Test-Driven Development (TDD) in learning contexts where generative AI tools are available as part of the development process. The research protocol is applied twice, resulting in two independent analyses conducted under the same protocol but in distinct instructional contexts. The first analysis is based on data collected during a short course conducted in a C programming environment, while the second analysis follows the same procedures using data from a separate short course conducted in JavaScript with the Jest testing framework.

3.1 Research Context and Participants

The quasi-experiment was conducted in the context of introductory programming education at two Brazilian public institutions, involving students at different stages of formal computing education. Across both instructional contexts, participants had already been introduced to basic programming concepts but were still in the early stages of developing systematic software engineering practices.

To ensure a common technical baseline across both instructional contexts, all participants completed a dedicated training session prior to the programming activities. This training covered the fundamental principles of Test-Driven Development, including the red–green–refactor cycle, incremental design, and the role of tests as executable specifications. Participants also received guidance on responsible use of generative AI tools, including formulating effective prompts, validating AI-generated outputs, and recognizing limitations such as hallucinations or overly generic responses. No prompt template was prescribed at any moment: students constructed their prompts freely. Prompt variability was therefore not controlled, but characterised in the thematic analysis of the interaction logs (Section 3.4).

The study comprises two independent analyses, each corresponding to a distinct short course. The first analysis is based on data collected during an introductory short course conducted using the `C` programming language and `assert library`, offered to undergraduate students enrolled in computing-related programs. In this context, the programming problems were selected based on their Cyclomatic Complexity and organized into two balanced lists, each containing tasks with equivalent structural complexity, as presented in Table 1 and Table 2.

All problems were designed to exhibit a cyclomatic complexity (CC) value of 3, ensuring uniform structural difficulty across rounds within the short course [15]. Cyclomatic complexity was operationalized as the number of decision points plus one, following McCabe's formulation. Decision points were defined as control-flow constructs introducing branching. The CC values were manually verified based on the control-flow structure of each reference solution, assuming a single connected component (P = 1).

Table 1: Problem List A (Assert 1).

ID	Program	Brief Description	CC
P01	GCD Calculator	Computes the GCD of two positive integers.	3
P02	Sum of Digits of a Number	Sums all digits of an integer.	3

Table 2: Problem List B (Assert 2).

ID	Program	Brief Description	CC
P03	Vowel Counter	Counts the vowels in a text string.	3
P04	Celsius/Fahrenheit Converter	Converts between Celsius and Fahrenheit.	3

A similar instructional structure was adopted for the second short course, which was conducted in a `JavaScript` environment using the `Jest` testing framework and offered to students enrolled in a technical secondary education program. This short course followed the same instructional structure as the first; however, students in the technical secondary education program were granted an additional 30 minutes during the problem-solving phase, in order to accommodate their educational level while preserving the overall task conditions. As in the first analysis, a distinct set of programming problems was defined based on cyclomatic complexity and organized into two balanced lists, each composed of three tasks with equivalent structural complexity, as presented in Table 3 and Table 4. All tasks also exhibit a cyclomatic complexity value of 3, supporting internal comparability while preserving methodological alignment between the two analyses.

Table 3: Problem List A (Jest 1).

ID	Program	Brief Description	CC
P01	GCD Calculator	Computes the GCD of two positive integers.	3
P02	Password Validator	Checks a password against validity rules.	3
P03	Sum of Digits of a Number	Sums all digits of an integer.	3

Table 4: Problem List B (Jest 2).

ID	Program	Brief Description	CC
P04	Prime Number Checker	Checks whether an integer is prime.	3
P05	Vowel Counter	Counts the vowels in a text string.	3
P06	Celsius/Fahrenheit Converter	Converts between Celsius and Fahrenheit.	3

3.2 Research Design Overview

This design integrates quantitative and qualitative evidence to characterise how generative AI shapes the application of TDD in educational contexts.

Figure 1 illustrates the overall research workflow adopted in this study. The workflow reflects the quasi-experimental structure and its complementary qualitative component, and is composed of four main stages: (i) task execution, (ii) data collection, (iii) quantitative analysis, and (iv) qualitative analysis. This same methodological workflow was applied independently in two instructional contexts, corresponding to the two short courses described in the previous section.

In the task execution stage, students individually performed programming activities following the TDD cycle, writing tests before implementation and iteratively refining their solutions. Depending on the instructional context, students were allowed to employ ChatGPT as a generative AI assistant during the development process. Students accessed ChatGPT via the standard web interface (chat.openai.com) without creating a user account; in this

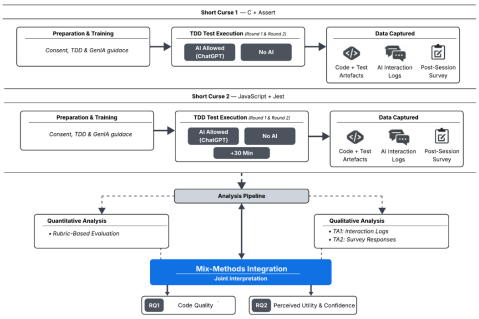


Figure 1: Quasi-experimental and qualitative study workflow.

configuration, the model served by default was GPT-4o mini, the version available to unauthenticated users at the time of both short courses (October 2025). In the second short course, conducted with technical secondary education students, participants were granted an additional 30 minutes during the problem-solving phase and worked on one additional programming problem when compared to the first short course. Throughout this stage, source code, test artefacts, and AI interaction logs were systematically collected.

The data collection stage focused on organizing and consolidating all empirical materials generated during development, including source files, test artefacts, AI conversation logs, and responses to post-session perception surveys. This procedure was conducted separately for each short course, preserving the integrity and contextual characteristics of the collected datasets.

The analysis stage integrated quantitative and qualitative procedures following an explicit evidence-chain rationale. Quantitative analyses operationalised through rubric-based scores characterise what was produced, describing the technical quality of submitted artefacts across the three retained metrics. Qualitative analyses comprising thematic examination of AI interaction logs and post-activity survey responses explain how students engaged with the AI tool during development and how they perceived and reflected on that experience. These two evidentiary layers are designed to be complementary: quantitative patterns establish the observable outcomes, while qualitative findings contextualise and explain those patterns in terms of student behaviour and perception. Both analyses were carried out independently for each short course, following the same evaluation criteria and analytic procedures.

To ensure transparency, all anonymized datasets, test suites, source code, and other artefacts used in the study are publicly available in an open GitHub repository.

3.3 Quantitative Analysis

The quantitative component of this study focused on assessing the quality of the code artefacts produced by participants through a structured and reproducible evaluation process. To guide this assessment, we adopted a rubric model inspired by the evaluation framework proposed by Stegeman et al. [22] for introductory programming courses. The original rubric includes four metrics: Correctness, Design, Readability, and Documentation.

Because the programming tasks were intentionally simple and short, any added documentation would likely be superficial and would not help discriminate code quality across submissions, so the Documentation metric was removed. The remaining metrics: Correction, Design/Structure, and Style/Readability were retained without altering their conceptual definitions, as they align directly with the competencies of our article. Each metric was evaluated on a 0–3 scale, where higher values indicate better adherence to the expected criteria. A Total score between 0–9 was computed as the sum of the three metrics. The rubric can be viewed in Table 5.

Table 5: Rubric for code quality evaluation used in this study.

Metric	1 – Insufficient	2 – Basic	3 – Good/Excellent	Max.
Correction	Program does not compile or fails on most basic tests; core logic is incorrect.	Program passes the main tests but still fails on some edge cases.	Program passes all provided tests, including edge cases; logic is robust.	/3
Design/Structure	Logic is concentrated in a single block; no meaningful use of functions; high code duplication.	Some functions are used, but separation of responsibilities is unclear or could be improved.	Code is modularised into small functions with well-defined responsibilities and a clear structure.	/3
Style/Readability	Variable and function names are vague; indentation and formatting are inconsistent or missing.	Names are understandable but do not follow a consistent pattern; formatting is acceptable but with minor issues.	Names are descriptive and consistent; code is well formatted and easy to read.	/3
Total	Total sum of the three metrics above.			/9

The unit of analysis is the *artefact*: the complete solution of one problem by one participant, whose form depends on the testing technology. In the C course, it is a single `.c` file with the implementation, the test function with the `assert` calls, and the `main` that executes it, in the same compilation unit. In the Jest course, it is the pair required by the framework: the implementation (`.js`) and its suite (`.test.js`). Therefore, the number of artefacts per participant is uniform within each course and differs only between courses.

The quantitative evaluation procedure described above is applied independently to each of the two short courses considered in this study. For each instructional context, code artefacts are analysed separately, using the same rubric, metrics, and data processing steps.

3.4 Qualitative Analysis

To investigate how students engaged with ChatGPT during the programming tasks, we conducted a qualitative analysis grounded in reflexive Thematic Analysis (TA) following the six-phase framework proposed by Braun and Clarke [4].

The qualitative analysis in this study is composed of two complementary thematic analyses, each applied to a different dataset. The first is the analysis of interaction logs (TA1), a thematic analysis of the conversation logs generated while students interacted with ChatGPT during the programming tasks. An interaction log is defined as the student's prompt plus the corresponding AI output. These logs reflect students' in-situ behaviors, doubts, problem-solving strategies, and immediate uses of the AI tool

The second is the analysis considering the survey responses (TA2), which consists of the open-ended responses from the post-activity survey, which included four free-text questions asking students to describe: (i) in what ways the AI helped them solve the problems; (ii) what they found easiest in the exercises; (iii) what they found most difficult; and (iv) any additional comments about the experience. These responses provide a broader perspective from a larger number of participants, capturing students' reflections, perceptions, and subjective evaluations of the activity. Both TA1 and TA2 were conducted independently for each of the two short courses considered in this study.

Each dataset was analysed separately to preserve its instructional context, but the analytic steps were identical across datasets. The coding and theme development were led by one member of the research team, and were iteratively discussed with at least one additional researcher, who reviewed the evolving codes and themes to support consistency and clarity of interpretation. Consistent with reflexive TA, which prioritises conceptual depth and pattern interpretation over numerical frequency [4], raw citation counts per code are not reported; the thematic results instead emphasise the relative conceptual weight of each theme.

3.5 Ethical Considerations

Participants provided digital informed consent prior to participation. No identifiable data was collected; only anonymized AI interaction logs were recorded and publicly shared for scientific transparency.

4 Results

This section presents the empirical findings: first, the quantitative evaluation of the code artefacts using the rubric; second, the qualitative thematic analysis of students' interactions with the AI and their perceptions of utility, difficulty, and confidence.

4.1 Rubric-Based Quantitative Analysis

For each short course, descriptive statistics characterise score distributions across metrics, complemented by graphical representations of variability and concentration.

4.1.1 Short Course 1 (Assert, C). The first quantitative analysis is based on the artefacts collected during the short course conducted using the C programming language and assert-based tests. Although a larger number of students participated in the instructional activities, only a subset produced complete and valid artefacts suitable for quantitative evaluation. As a result, the analysis includes artefacts produced by four participants in Group 1 and one participant in Group 2, whose submissions consistently demonstrated the application of the TDD methodology.

Table 6 summarises the descriptive statistics obtained from the rubric-based evaluation of artefacts produced in the Assert short course. We report the results as an overall analysis within this instructional context, aggregating rubric scores across all validated submissions.

Table 6: Descriptive statistics (Assert, overall).

Statistic	Correction	Design/Structure	Style/Readability	Total
Mean	1.95	2.20	1.80	5.95
Median	2.00	2.50	2.00	6.50
Std	0.89	1.01	1.01	2.72
Min	0.00	0.00	0.00	0.00
Max	3.00	3.00	3.00	9.00

The descriptive statistics indicate intermediate-to-high central tendencies across the retained metrics, combined with non-trivial dispersion. This suggests heterogeneous outcomes among the analysed submissions, in which some artefacts achieved strong performance on the rubric while others exhibited partial or inconsistent attainment. As a consequence, total scores span a substantial portion of the rubric scale, reflecting variability in overall artefact quality within the short course context.

Across the analysed artefacts, Correction exhibits a distribution centred around intermediate-to-high values, while still showing a noticeable spread across the scale. Several submissions approach the upper bound of the rubric, but lower scores are also present, indicating that functional correctness was achieved to varying degrees among participants. This dispersion is coherent with differences in how consistently the TDD cycle was applied to ensure fully correct behaviour across cases.

For Design/Structure, the distribution appears more concentrated toward higher values, suggesting that, even when functional aspects were not fully consolidated, participants often produced solutions with reasonably coherent structural organisation. In contrast, Style/Readability displays a broader spread, with scores extending across a wider range of the rubric. This pattern indicates greater inconsistency in stylistic and readability-related aspects of the code, which were less uniformly addressed across submissions.

In addition to the descriptive statistics, we analysed the distribution of rubric scores using a Likert-style representation. The approach preserves the structure of the rubric by mapping each metric individually to four ordered agreement levels. Specifically, each original 0–3 score was converted as follows: 0 = Totally disagree, 1 = Disagree, 2 = Agree, and 3 = Totally agree. For each metric, rubric instances were counted within each Likert level and subsequently normalised to obtain the percentage distributions illustrated in Figure 2.

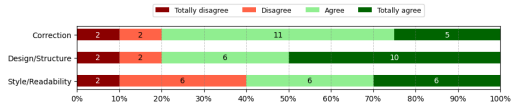


Figure 2: Rubric score distribution for the Assert course.

This transformation offers a complementary perspective on rubric performance by revealing how scores are distributed across qualitative agreement levels rather than aggregated numeric values. Viewed through this lens, Correction emerges as the most stable dimension, with the majority of instances concentrated in the higher agreement categories. This indicates that, across analysed artefacts, functional requirements were generally met with reasonable consistency.

In contrast, Design/Structure and Style/Readability exhibit a more diffuse distribution across the Likert scale. Scores for these dimensions span multiple agreement levels, suggesting greater heterogeneity in how participants addressed structural organisation, naming conventions, and code formatting. Rather than indicating uniformly weak performance, this dispersion reflects uneven consolidation of these aspects across submissions.

4.1.2 Short Course 2 (Jest, JavaScript). The second quantitative analysis is based on the artefacts collected during the short course conducted using JavaScript and Jest tests. The analysis includes three complete artefact sets whose submissions consistently demonstrated the application of the TDD methodology. All analysed artefacts correspond to final submissions produced at the end of the instructional activities. Each artefact was evaluated using the rubric described previously, resulting in scores for the three retained metrics (Correction, Design/Structure, and Style/Readability) and a total score computed as their sum.

Table 7 summarises the descriptive statistics obtained from the rubric-based evaluation of artefacts produced in the Jest short

course. We report the results as an overall analysis within this instructional context, aggregating rubric scores across all validated submissions. The table provides an overview of central tendency and dispersion for each retained metric, as well as for the total score.

Table 7: Descriptive statistics (Jest, overall).

Statistic	Correction	Design/Structure	Style/Readability	Total
Mean	1.89	2.22	2.22	6.33
Median	2.00	2.00	2.00	6.00
Std	0.58	0.73	0.81	1.75
Min	1.00	1.00	1.00	4.00
Max	3.00	3.00	3.00	9.00

Across the analysed artefacts, the statistical profile of the Jest short course is marked by moderate-to-high central tendencies accompanied by visible dispersion. Scores related to functional correctness are predominantly located around intermediate values of the rubric scale, indicating that solutions generally achieved acceptable behaviour, although full consolidation was not uniformly observed. This pattern is complemented by the non-functional metrics, where structural and stylistic aspects also exhibit variability, reflecting differences in how consistently organisational and readability-related aspects were addressed across artefacts.

Consistent with this profile, total scores span a meaningful range of values rather than converging tightly near the upper bound of the rubric scale. This distribution suggests heterogeneous levels of overall artefact quality within this instructional context, highlighting the coexistence of partially consolidated and more refined submissions among the analysed artefacts.

A salient characteristic of the distributions is that functional correctness tends to cluster around intermediate values, with occasional lower and higher instances. This suggests that while many solutions achieved acceptable behaviour, a smaller number either failed to meet key functional requirements or reached stronger levels of consolidation. In the non-functional metrics, Design/Structure and Style/Readability show score ranges that extend across adjacent levels and often reach the upper portion of the rubric, indicating that organisational and readability-related decisions were frequently addressed but with uneven consistency across artefacts.

The distribution of rubric scores was also examined using a Likert-style representation, following the same conversion procedure adopted for the Assert short course. As previously described, rubric scores were mapped to ordered agreement levels, and the resulting distributions were normalised to obtain the percentage values illustrated in Figure 3.

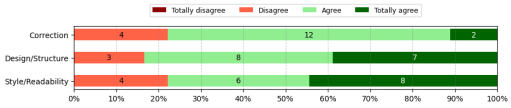


Figure 3: Rubric score distribution for the Jest course.

From this perspective, the Likert-scale visualisation reveals how scores are distributed across qualitative agreement levels rather than numerical ranges. For the Jest short course, the distributions do not converge strongly toward a single agreement level. Instead, scores are spread across adjacent levels, particularly within the Agree and Disagree ranges, reflecting moderate consolidation of the evaluated metrics.

This pattern is especially visible in the non-functional metrics. Design/Structure and Style/Readability exhibit distributions that extend across multiple agreement levels, suggesting that organisational and stylistic aspects of the code were addressed with varying degrees of consistency. While higher agreement levels are present, they do not dominate the distributions, indicating that these metrics were not uniformly consolidated across artefacts.

4.2 Qualitative Results of Thematic Analyses

We first analyse the open-ended survey responses, which capture participants' retrospective perceptions (Section 4.2.1), and then the interaction logs, which capture in-situ behaviours, difficulties, and problem-solving strategies (Section 4.2.2). The same protocol was applied to each course.

4.2.1 Thematic Analysis of Survey Responses. The thematic analysis of the open-ended survey responses complemented the log-based findings by providing insight into students' perceptions, interpretations, and self-reported experiences after completing the programming tasks with ChatGPT. This analysis is based on 15 open-ended survey responses collected from participants in the C short course. While the interaction logs captured how students behaved during the activity, the survey responses revealed how they understood and evaluated their own use of the AI tool. Table 8 summarizes the thematic structure derived from the open-ended survey responses by presenting the four consolidated themes and their associated codes, serving as a reference for interpretation and reinforcing the transparency of the analytic procedure.

Table 8: Thematic analysis of survey responses (Assert).

Theme	Codes
Type of assistance Students Perceived from the AI (30,3%)	AI helped by generating the function/code; AI helped with logic understanding; AI helped recalling mathematical concepts; AI helped identifying usable functions; AI clarified problem understanding; AI explained TDD; AI assisted with test construction; AI sped up task completion; AI solved the entire exercise for the student; AI reduced doubts / increased clarity.
What was easiest? (24,2%)	Tests/asserts were easy; Logic became easy after understanding; Exercise was simple / basic; Math operations were easy; Using the AI made things easy; Writing the code after logic was easy.
What was most difficult? (22,7%)	Difficulty formulating the function/algorithm; Difficulty creating tests first (TDD challenge); Difficulty recalling basic math or C concepts; Difficulty making the code run; Difficulty with logic; Difficulty with file structure (.h, .c, main); Difficulty using/assert correctly; Difficulty communicating with the AI; Difficulty with string manipulation; "Everything was difficult".
Additional comments (22,7%)	No comments; Misunderstanding of instructions; Enjoyment/Interest in TDD; Strong positive evaluation of AI.

Types of assistance students perceived from the AI capture how they described the concrete ways in which ChatGPT supported their problem solving. Responses such as "It helped me remember the concepts of GCD." or "It created the function needed for the problem and helped me understand it." emphasized that the AI helped generate functions or complete solutions, clarify logical reasoning, recall mathematical concepts, explain TDD, support test construction, accelerate workflow, or reduce uncertainty. These statements reveal how students perceived ChatGPT not merely as a code generator, but as a broad instructional aid that shaped their understanding and approach to the tasks.

The second theme concerns the elements that students perceived as facilitating their progress during the activity. Several responses indicate that once the underlying logic of a solution was understood,

Table 9: Thematic analysis of survey responses (Jest)

Theme	Codes
AI as procedural and conceptual scaffold (44,4%)	AI supported TDD execution and logic formulation; AI helped clarify understanding; AI facilitated access to formulas/rules; AI used as a reliable research/reference source.
Prompting as skill and friction point (22,2%)	Prompt formulation perceived as easy; Difficulty specifying intent to the AI.
Human-in-the-loop verification of AI output (11,1%)	Correcting/validating the AI output was perceived as easy or central to the task.
Residual difficulty and reasoning continuity (22,2%)	No particular difficulty reported; Difficulty maintaining logic at specific moments.

subsequent implementation became relatively straightforward. Others highlighted the simplicity of the assigned tasks or the familiarity of the required mathematical operations. A notable subset of students emphasized that the presence of ChatGPT itself contributed to this sense of ease, either by clarifying intermediate steps or by reducing the cognitive effort required to structure the solution. As one student reported: "Using the AI made things easy.".

Reported difficulties encompassed both conceptual and technical aspects, including formulating algorithms, recalling foundational mathematical or C programming concepts, ensuring that the program executed properly, and working with language-specific structures such as header files and separate compilation units. Some students reported uncertainty regarding the correct use of assert or the sequencing required by TDD, while others identified obstacles related to logical reasoning or string manipulation. One student noted: "Remembering basic math and C concepts was difficult.". A few responses expressed more generalized difficulty, indicating that multiple aspects of the exercise were simultaneously challenging. These difficulties are consistent with patterns observed in the behavioral log data, reinforcing the presence of persistent gaps in prerequisite knowledge and suggesting areas where additional pedagogical support may be required.

The final theme synthesizes students' overall impressions and evaluative reflections regarding the activity. Some participants expressed enthusiasm toward the use of TDD or positively assessed the usefulness of ChatGPT as a support tool. As reported: "Extremely interesting to use TDD." and "ChatGPT is awesome.". A small number of responses indicated confusion about specific instructions, particularly regarding the requirement to employ TDD, suggesting opportunities for improving task communication in future iterations.

Following the same qualitative protocol, we conducted a thematic analysis of the open-ended survey responses collected in the Jest short course. In this instructional context, the analysis draws on 3 open-ended survey responses provided by participants in the JavaScript (Jest) short course. Responses to the three questions were treated as coded segments, enabling the construction of cross-cutting themes that span students' reported support, perceived ease, and perceived difficulty when working with AI during the activity. Four themes emerged, and their distribution is reported in Table 9.

AI as procedural and conceptual scaffold (44,4%), captures how students described AI as supporting both the execution of the activity and their understanding of underlying concepts. This includes references to AI helping with TDD and logic, clarifying understanding, and providing formulas or rules as a form of guidance. For instance, one student stated: "By doing the TDD and the logic", indicating that the model was perceived as assisting with both process and reasoning.

Table 10: Table of the thematic analysis process (Assert).

Theme	Codes
Conceptual Difficulties in Learning Programming and TDD (33,3%)	Doubt about problem/algorithm formulation; Request for conceptual explanation; Request for numerical/example-based explanation; Doubt about mathematical concepts; Doubt about C programming concepts.
Dependence on the AI for Code Generation, Correction, and Refinement (20,0%)	Request for complete code solution; Request for isolated function or constrained output; Use of AI as a ready-made answer generator; Debugging student-written code; Correction of logical errors; Correction of syntax/compilation errors; Refactoring or restructuring guided by the AI; Request for alternative implementation; Request for simplified version; Request for code re-sending or reformatting.
Use of the AI as Support for Tests, Asserts, and TDD (33,3%)	Doubt about TDD/testing; Doubt about the use of assert; Request for explanation about assert; Request for test case examples; Request for TDD implementation; Request for assert implementation.
Interaction Strategies and Adaptation of AI Behavior (13,3%)	Sending test examples in the prompt; Persona request.

Prompting as skill and friction point with 22,2%, reflects the centrality of prompt formulation in students' experience. While prompt writing was sometimes framed as easy, students also highlighted difficulties in communicating intent precisely to the model. This is illustrated by the response: "Showing the chat what I want", which points to prompt specification as an interactional challenge even when the model is considered helpful.

The third, Human-in-the-loop verification of AI output (11,1%), highlights an evaluative stance in which students positioned themselves as actively checking and correcting AI responses. Rather than viewing AI output as final, one student described the easiest aspect as: "Correcting the AI", suggesting that verification and adjustment of generated output became a salient part of task completion.

Finally, Residual difficulty and reasoning continuity (22,2%) captures the remaining challenges reported by students, ranging from the absence of perceived difficulty to momentary breakdowns in maintaining the solution logic. One participant reported: "So I found it more difficult to have the logic at some specific moment", indicating that sustaining algorithmic reasoning throughout the task remained challenging even with AI assistance.

4.2.2 Thematic Analysis of Interaction Logs. The thematic analysis of the interaction logs revealed four overarching themes that characterize how students engaged with ChatGPT during the programming tasks in the first (Assert) short course. These themes describe recurring patterns of behavior related to conceptual understanding, reliance on the AI for code generation and correction, support for testing and TDD, and strategic prompting. Table 10 provides a summary of this theme structure, illustrating the relationship between the individual categories (codes) identified during the analysis and the themes into which they were grouped. The table also depicts how these categories emerged from the collected ChatGPT interaction logs, highlighting the flow from raw data to codes and finally to higher-level themes. In total, the distribution of coded segments across the themes was as follows: 33,3% related to conceptual difficulties, 20,0% to dependence on AI for coding support, 33,3% to assistance with tests and TDD, and 13,3% to prompting strategies.

The first theme, Conceptual Difficulties in Learning Programming and TDD, reflects students' frequent attempts to resolve foundational misunderstandings related to mathematics, C programming constructs, problem-to-algorithm formulation, and the conceptual foundations of TDD and assert. Students often used ChatGPT to obtain conceptual explanations and numerical examples, suggesting that the model served as a source of didactic

Table 11: Table of the thematic analysis process (Jest).

Theme	Codes
Dependence on the AI for Code Generation, Correction, and Refinement (33,9%)	Request for complete program solution in JavaScript; Request for reusable function implementation; Use of AI as a ready-made answer generator; Debugging student-written code; Correction of logical errors; Refactoring for cleaner structure; Request for simplified solution; Request for alternative implementation.
Conceptual Difficulties in Programming and Algorithmic Reasoning (25,4%)	Doubt about prime-number reasoning; Doubt about algorithmic limits; Request for formula-based explanation; Request for numerical/example-based explanation; Doubt about mathematical concepts underlying algorithms; Doubt about programming concepts.
Use of the AI as Support for Tests, Jest, and TDD (22,0%)	Request for Jest test suite creation; Request for TDD-oriented guidance; Request for edge cases and boundary tests; Request for test patterns and assertions in Jest; Request for test scaffolding and file structure; Request for test coverage across multiple scenarios.
Interaction Strategies and Prompt Adaptation (18,6%)	Persona/role framing; Prompt constraints; Iterative prompting for refinement; Providing partial code or snippets to steer the AI.

scaffolding during the tasks. For instance, one participant explicitly requested clarification on numerical reasoning behind digit extraction: “explain better why n%10 gets the last digit, with an example.”.

The second theme, Dependence on the AI for Code Generation, Correction, and Refinement, captures how students repeatedly relied on the model to produce executable code, identify and correct errors, propose simpler alternatives, or revise previously generated solutions. Several interactions exhibited iterative cycles in which students refined their solutions through repeated requests, indicating sustained reliance on the AI as a problem-solving engine. As an example, a participant directly asked the model to generate a solution scaffold: “help me create a C function to compute the greatest common divisor (GCD).”.

The third theme, Use of the AI as Support for Tests, Asserts, and TDD, encompasses interactions where students explicitly sought help with constructing test cases, understanding assertions, or applying TDD principles. Requests for test examples, explanations of testing constructs, and direct assistance in generating or validating tests were common, positioning ChatGPT as a technical assistant for test-related aspects of the activity. For example, one participant asked for practical guidance on assertions: “how to use assert, use it in a function, and include two values.”.

Interaction Strategies and Adaptation of AI Behavior includes cases where students attempted to guide or condition the model’s behavior through specific prompt strategies such as persona requests or by providing curated test cases as examples. Although less frequent, these behaviors demonstrate students’ emerging understanding of how prompt design can influence AI responses. One interaction illustrates this strategy through role framing: “you are a senior C programmer...”.

Building on this perspective, the interaction logs from the second short course (JavaScript with Jest) were analysed using the same thematic procedure, resulting in a comparable four-theme structure that captures how students engaged with ChatGPT during the programming tasks. Table 11 summarises the relationship between the coded categories identified in the logs and the higher-level themes constructed for this instructional setting. In total, the distribution of coded segments across themes was as follows: 33,9% related to conceptual difficulties, 25,4% to dependence on the AI for coding support, 22,0% to assistance with tests and TDD in Jest, and 18,6% to interaction strategies and prompt adaptation.

In the Jest short course, the first theme, Dependence on the AI for Code Generation, Correction, and Refinement (JS/Jest), reflects interactions in which students relied on ChatGPT to produce executable JavaScript solutions and iteratively adjust them. Prompts frequently involved requests for complete implementations or function-level code, followed by refinement cycles such as debugging, restructuring for clarity, and exploring simplified or alternative approaches. For example, one participant asked the model to critically assess their solution: “what are the main flaws in the code.”.

The Conceptual Difficulties in Programming and Algorithmic Reasoning, captures difficulties related to the reasoning required to operationalise solutions, especially in tasks involving numerical properties and algorithmic constraints (e.g., prime checking logic and divisor bounds). In these interactions, students used the model as an explanatory resource through requests for conceptual clarification, justification of algorithmic decisions, and numerical examples. A clear illustration appears when a participant asked: “104729 how would you determine whether this number is prime?”.

Use of the AI as Support for Tests, Jest, and TDD, groups interactions where students explicitly sought assistance in constructing Jest unit tests and aligning their work with a TDD workflow. Requests included generating test suites, identifying boundary and invalid cases, selecting appropriate assertions and matchers, and clarifying how to structure test files to support incremental development. For instance, a participant explicitly requested test generation in Jest: “you are a programmer with 20 years of experience; I need you to create tests... using... JavaScript... the Jest framework.”.

Finally, Interaction Strategies and Prompt Adaptation includes instances where students attempted to steer the model’s responses through role framing, prompt constraints, iterative follow-up requests, or by providing partial code snippets for critique and correction. Although less frequent, these interactions indicate an emerging awareness that prompt formulation can shape the specificity and usefulness of AI outputs in a Jest-based development setting. One such role-framing prompt begins with: “hi chat, I want you to act like a university student...”.

5 Discussion

This section interprets the findings of the study in light of the research questions and relevant literature on generative AI in programming education.

5.1 Overview

Across both instructional contexts analysed in this quasi-experiment, a consistent pattern emerged: students were generally able to achieve functional solutions with AI support, while non-functional aspects of code quality (especially design/structure and style/readability) remained more uneven across submissions. This pattern is visible in the rubric-based evaluation, where correctness tends to concentrate at higher levels than the metrics associated with modularisation, naming clarity, and formatting. The Likert-style representations further reinforce that dispersion is more pronounced in structural and stylistic metrics than in functional correctness, suggesting that meeting functional requirements may stabilise earlier than internalising broader quality conventions.

The qualitative analyses help explain how these quantitative patterns materialised during task execution. Interaction logs show that students frequently treated the AI as an active collaborator in the problem-solving process, using it to generate solution scaffolds, debug code, refine implementations, and obtain conceptual clarification. At the same time, both datasets exhibit evidence of prompting-related strategies and friction, indicating that students' ability to articulate intent and constrain AI output shaped the usefulness of the assistance received. Survey responses complement these observations by highlighting that students perceived AI as a procedural and conceptual scaffold, but also reported difficulties related to communicating requirements and sustaining reasoning at specific moments. Taken together, these three sources of evidence form a coherent evidence chain: rubric scores establish what was produced, interaction logs explain how students engaged with the AI tool and survey responses reveal how they reflected on that engagement.

5.2 RQ1: Technical Characteristics of Code Produced under AI-Assisted TDD

Across both instructional contexts analysed (Assert/C and Jest/JavaScript), the quantitative results indicate that participants were generally able to reach functionally correct solutions with AI support, with the Correction metric showing the lowest dispersion across submissions. This pattern is consistent with prior research showing that LLMs can reduce syntactic friction and accelerate progress toward executable outputs in introductory programming tasks [7, 8, 19]. In our datasets, this functional adequacy is also reflected in the interaction logs, where students repeatedly engaged the model to validate reasoning, resolve errors, and obtain direct implementations, resembling the iterative AI-mediated refinement cycles reported by Mock et al. [16] and the grounded assistance behaviours observed by Barke et al. [2].

However, the rubric outcomes also show that achieving correct behaviour does not automatically translate into consistent non-functional quality. In both contexts, Design/Structure and Style/Readability exhibited greater dispersion than Correction, suggesting that modularisation, naming discipline, and formatting conventions were consolidated unevenly across submissions. Similar tendencies have been reported in introductory settings in which learners use generative AI primarily as a code-producing aid: students often reach solutions faster, but structural habits associated with software engineering practice remain less stable [6, 10, 12]. In our study, this gap is compatible with the short duration and task-focused nature of the minicourses: while AI support may facilitate completion, it does not inherently enforce systematic structure or stylistic consistency.

5.3 RQ2: Novice Programmers' Perceptions of Utility and Challenges in AI-Assisted TDD

The qualitative evidence from the post-activity surveys indicates that students consistently framed generative AI as a form of procedural and conceptual support while working with TDD. Across both instructional contexts, participants described AI as helping them operationalise task logic, clarify understanding, and access rules or formulas more efficiently, which aligns with prior findings that LLMs are often perceived as a scaffold for both explanation and implementation in novice programming settings [8, 19]. In our data, this perceived utility is coherent with the interaction logs, where students repeatedly used the model to obtain step-by-step clarifications, validate reasoning, and refine solutions during short time-bounded tasks.

At the same time, survey responses highlight that the experience of AI assistance does not eliminate difficulty; rather, it redistributes it. A recurring point of friction concerns communication with the model, where students explicitly reported challenges related to specifying intent and expressing what they wanted the AI to produce. This perceived difficulty maps directly onto the prompting behaviours observed in the logs, including role framing, constrained requests, and iterative follow-up prompts, suggesting that effective coordination with the AI became a skill component of the activity. This resonates with research emphasising prompting as an emergent literacy in AI-supported learning environments, in which the quality of assistance is strongly shaped by how users articulate constraints, context, and verification expectations [16]. A similar spectrum was reported by Ardiç et al. [1], whose students either requested complete test suites in a single prompt or refined outputs through iterative prompting.

The Perceptions of difficulty and confidence appear intertwined with moments where students struggled to sustain reasoning continuity. Even when AI was viewed as helpful, some respondents reported difficulty in maintaining the logic at specific points of the task. This aligns with concerns raised in prior work about the risk of shallow progress or fragmented reasoning when learners rely heavily on generated code without fully consolidating underlying problem-solving steps [6, 12].

5.4 Implications

The findings suggest that LLMs can effectively reduce entry barriers for novices, particularly in supporting mathematical reasoning, syntax, and debugging. Educators may leverage the tool as a conceptual tutor, especially in early-stage programming tasks. However, structural skills (modularity), naming, formatting require targeted instructional reinforcement, as AI-generated solutions do not reliably promote these competencies.

AI assistance can support the construction of tests, but may inadvertently shift students away from the "test-first" discipline if used indiscriminately. Structured prompts, activity scaffolds, or controlled constraints may be necessary to ensure that learners maintain focus on the TDD cycle rather than delegating test design to the model. This is supported by prior evidence showing that strategy-guided prompts yielded more effective AI-generated test cases [21], whereas full delegation was associated with a diminished sense of ownership over tests [1].

Given the observed patterns of dependence, educators should emphasize responsible use, critical evaluation of AI-generated code, and autonomy before automation. Similar to recommendations from Reihanian et al. [20], Prather et al. [19], and Khan et al. [9], AI literacy should be embedded into programming curricula to develop students' capacity to use generative tools as collaborators rather than substitutes.

The quasi-experimental design with its complementary qualitative component adopted here highlights opportunities for

longitudinal studies examining how AI-supported TDD skills evolve over time and whether initial gains in correctness translate into sustainable improvements in software engineering practices.

5.5 Limitations

Our measures of artefact quality and learner experience may not capture all facets of TDD learning. The adapted three-metric rubric, though grounded in prior work, involves subjective judgment. Likewise, the short post-activity survey on perceived utility, difficulty, and confidence may not reflect learning outcomes or longer-term understanding. Thematic analysis is inherently interpretive; although coding and theme development were reviewed through iterative discussion within the research team, different analysts could emphasise different patterns. Finally, the interaction logs reflect only what participants recorded and may not represent all AI-related reasoning or off-platform interactions.

Contextual factors may also have influenced outcomes independently of AI assistance. The two instructional settings differed in participant profile, programming language, and testing framework, and the second short course also allowed additional time, which can affect performance and perceived difficulty. Despite supervision and a shared protocol, variations in pacing, prior knowledge, and individual problem-solving strategies may have acted as confounders. Notably, the extra 30 minutes given to technical students is a limitation. Comparing the qualitative analyses of both contexts, we found no marked differences: requests for complete solutions, conceptual doubts, and AI support for building tests predominated in both. The most visible difference concerned prerequisites rather than performance: undergraduates reported the need to recall mathematical and C concepts, whereas technical students reported more difficulty specifying their intentions to the model. Still, the design does not isolate the effect of the extra time.

Generalisability is limited by the study context and task design. The validated dataset has few complete artefact sets, and the short minicourses may not reflect semester-long learning dynamics. The programming tasks were intentionally simple (cyclomatic complexity = 3), which may not elicit behaviours that emerge in more complex or multi-module TDD scenarios. Additionally, the study examined a single generative AI model (ChatGPT, GPT-4o mini); findings may not extend to other LLMs, interfaces, or contexts.

The small sample size and incomplete individual-level coverage limit the strength of quantitative conclusions. Results are primarily descriptive, and the study is not powered for robust inferential comparisons. Two evaluators assessed code quality, adding subjectivity that may limit comparative interpretation.

5.6 Contributions

This study provides empirical evidence on how novice programmers apply TDD when assisted by a generative AI tool, contributing to an area where systematic investigations remain limited.

The study also provides a structured research protocol and an experimental artefact package, combining rubric-based evaluation, assert-based test suites, interaction logs, thematic analysis, and perception data, that can be reused or adapted in future research on AI-supported programming practices.

In addition, the findings shed light on the role of ChatGPT as both a conceptual tutor and a test-construction assistant, complementing prior work on proactive testing and AI-assisted code refinement. The analysis of interaction logs further reveals emerging patterns of prompting, delegation, and dependence, contributing to broader discussions on human–AI collaboration in software development. Also, we discuss pedagogical implications for integrating generative AI into early TDD activities, emphasizing responsible use, structured guidance, and the need to balance automation with conceptual understanding.

6 Conclusion

This exploratory quasi-experiment, complemented by a qualitative component, examined how novice programmers apply Test-Driven Development when assisted by a generative AI tool, integrating evidence from code artefacts, behavioural traces, and perception data. The findings show that while students consistently achieved functional correctness, structural and stylistic aspects of their solutions remained heterogeneous. The behavioural analysis revealed that generative AI served not only as a source of code and test examples but also as a conceptual tutor supporting reasoning, clarification, and debugging. Students' perceptions echoed these findings, indicating that AI assistance reduced uncertainty, increased clarity, and contributed to a more accessible development experience.

Taken together, these results highlight both the pedagogical opportunities and the challenges associated with introducing generative AI into early-stage TDD learning environments. AI support can scaffold conceptual understanding, accelerate debugging, and assist in test construction, yet it may also encourage iterative delegation cycles that weaken students' engagement with test-first reasoning and structural refinement. These dynamics underscore the importance of designing learning activities that balance automation with human-centred reasoning, fostering responsible use and intentional scaffolding.

Future research may expand on these findings by examining AI-assisted TDD in more complex programming tasks, exploring longitudinal effects on students' reasoning skills, and comparing different generative models within controlled instructional designs. Additional studies may also investigate strategies for scaffolding test-first thinking when AI tools are present, as well as methods for mitigating over-reliance while preserving the benefits of conceptual support.

Use of Generative AI Tools

The authors used ChatGPT (OpenAI) for language revision, scripting, search query construction, and formatting. All outputs were verified by the authors, who accept full responsibility for the manuscript.

ARTIFACT AVAILABILITY

Artifacts are available at https://github.com/igormichalski/TDD-LLM-Education-Study.

ACKNOWLEDGMENTS

Supported by UEMS (PIBICin). We thank UEMS and IFMS for facility support, and UNAL for hosting researchers during data analysis.

REFERENCES

[1] Baris Ardiç, Quentin Le Dilavrec, and Andy Zaidman. 2026. How students use generative AI for software testing: An observational study. *Empirical Software Engineering* 31, 6 (2026), 167. doi:10.1007/s10664-026-10898-0

[2] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2022. Grounded Copilot: How Programmers Interact with Code-Generating Models. arXiv:2206.15000 [cs.HC] https://arxiv.org/abs/2206.15000

[3] Beck. 2002. *Test Driven Development: By Example.* Addison-Wesley Longman Publishing Co., Inc., USA.

[4] Virginia Braun and Victoria Clarke. 2006. Using thematic analysis in psychology. *Qualitative Research in Psychology* 3 (2006), 77–101. Issue 2. https://uwe-repository.worktribe.com/output/1043060

[5] Margarida Ferreira, Luis Viegas, Joao Pascoal Faria, and Bruno Lima. 2025. Acceptance Test Generation with Large Language Models: An Industrial Case Study. arXiv:2504.07244 [cs.SE] https://arxiv.org/abs/2504.07244

[6] Ronivaldo Ferreira, Victor H. S. Pinto, Cleidson R. B. de Souza, and Gustavo Pinto. 2024. Assisting Novice Developers Learning in Flutter Through Cognitive-Driven Development. arXiv:2408.11209 [cs.SE] https://arxiv.org/abs/2408.11209

[7] Philipp Haindl and Gerald Weinberger. 2024. Students' Experiences of Using ChatGPT in an Undergraduate Programming Course. *IEEE Access* PP (01 2024), 1–1. doi:10.1109/ACCESS.2024.3380909

[8] Majeed Kazemitabaar, Xinying Hou, Austin Henley, Barbara Ericson, David Weintrop, and Tovi Grossman. 2024. How Novices Use LLM-based Code Generators to Solve CS1 Coding Tasks in a Self-Paced Learning Environment. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1 (SIGCSE 2024)*. ACM, New York, NY, USA, 1–12. doi:10.1145/3626252.3630918

[9] Maryam Khan, Muhammad Azeem Akbar, and Jussi Kasurinen. 2025. Integrating LLMs in Software Engineering Education: Motivators, Demotivators, and a Roadmap Towards a Framework for Finnish Higher Education Institutes. arXiv:2503.22238 [cs.SE] https://arxiv.org/abs/2503.22238

[10] Yoonji Kim, Youngkyung Choi, Daye Kang, Minkyeong Lee, Tek-Jin Nam, and Andrea Bianchi. 2020. HeyTeddy: Conversational Test-Driven Development for Physical Computing. *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.* 3, 4, Article 139 (Sept. 2020), 21 pages. doi:10.1145/3369838

[11] Aayush Kumar, Daniel Prol, Amin Alipour, and Sruti Srinivasa Ragavan. 2025. Web vs. LLMs: An Empirical Study of Learning Behaviors of CS2 Students. doi:10.48550/arXiv.2501.11935

[12] W. Lyu, Y. Wang, Y. Sun, and Y. Zhang. 2025. Will Your Next Pair Programming Partner Be Human? An Empirical Evaluation of Generative AI as a Collaborative Teammate in a Semester-Long Classroom Setting. In *Proceedings of the Twelfth ACM Conference on Learning @ Scale*. ACM, New York, NY, USA, 83–94. doi:10.1145/3698205.3729544

[13] Shuai Ma, Junling Wang, Yuanhao Zhang, Xiaojuan Ma, and April Yi Wang. 2025. DBox: Scaffolding Algorithmic Programming Learning through Learner-LLM Co-Decomposition. arXiv:2502.19133 [cs.HC] https://arxiv.org/abs/2502.19133

[14] Md Motaleb Hossen Manik. 2025. ChatGPT vs. DeepSeek: A Comparative Study on AI-Based Code Generation. arXiv:2502.18467 [cs.SE] https://arxiv.org/abs/2502.18467

[15] T.J. McCabe. 1976. A Complexity Measure. *IEEE Transactions on Software Engineering* SE-2, 4 (1976), 308–320. doi:10.1109/TSE.1976.233837

[16] M. Mock, J. Melegati, and B. Russo. 2025. Generative AI for Test Driven Development: Preliminary Results. In *Product-Focused Software Process Improvement (PROFES 2024)*. Springer Nature Switzerland, Cham, 24–32. doi:10.1007/978-3-031-72781-8_3

[17] Michael Muñoz, Juan Torre, Stefany López, Stiven Herrera, and Christian Candela. 2024. Comparative Study of AI Code Generation Tools: Quality Assessment and Performance Analysis. *LatIA* 2 (07 2024), 104. doi:10.62486/latia2024104

[18] Muhammet Mustafa Olmez and Edward Gehringer. 2024. Automation of Test Skeletons Within Test-Driven Development Projects. In *2024 36th International Conference on Software Engineering Education and Training (CSEE&T)*. 1–10. doi:10.1109/CSEET62301.2024.10663016

[19] James Prather, Juho Leinonen, Natalie Kiesler, Jamie Gorson Benario, Sam Lau, Stephen MacNeil, Narges Norouzi, Simone Opel, Vee Pettit, Leo Porter, Brent N. Reeves, Jaromir Savelka, David H. Smith IV, Sven Strickroth, and Daniel Zingaro. 2024. Beyond the Hype: A Comprehensive Review of Current Trends in Generative AI Research, Teaching Practices, and Tools. arXiv:2412.14732 [cs.CY] https://arxiv.org/abs/2412.14732

[20] Iman Reihanian, Yunfei Hou, Yu Chen, and Yifei Zheng. 2025. A Review of Generative AI in Computer Science Education: Challenges and Opportunities in Accuracy, Authenticity, and Assessment. arXiv:2507.11543 [cs.CY] https://arxiv.org/abs/2507.11543

[21] Saebom Son and Jiyoung Song. 2025. Evaluating the Educational Impact of AI-Generated Test Cases in a Test-Driven Development Environment for Python Beginners. *Smart Media Journal* 14, 9 (2025), 55–62. doi:10.30693/SMJ.2025.14.9.55

[22] Martijn Stegeman, Erik Barendsen, and Sjaak Smetsers. 2016. Designing a rubric for feedback on code quality in programming courses. In *Proceedings of the 16th Koli Calling International Conference on Computing Education Research* (Koli, Finland) *(Koli Calling '16)*. Association for Computing Machinery, New York, NY, USA, 160–164. doi:10.1145/2999541.2999555

[23] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. 2012. *Experimentation in Software Engineering*. Springer Publishing Company, Berlin, Heidelberg.

[24] Zhiqiang Yuan, Yiling Lou, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, and Xin Peng. 2023. No More Manual Tests? Evaluating and Improving ChatGPT for Unit Test Generation. arXiv:2305.04207 [cs.SE]